

NYEDArch

Not Your Everyday Archive

The NYEDArch Framework

Technical Documentation, Prototype Record & Evaluation Dossier

Self-protecting executable data capsules with cryptographically enforced multi-factor authorization.

Field	Value
Document type	Prototype technical record / IP disclosure support / evaluation dossier
Subject	NYEDArch Framework — Builder client and generated capsule runtime
Version	Prototype 0.0.1 · crypto version 1 · package format version 1
Implementation	Rust (9 workspace crates + desktop client)
Toolchain verified	Rust 1.91.1 (workspace rust-version 1.82)
Test status	162 automated tests · 0 failures · 0 compiler warnings
Status	Working prototype — full pipeline executed end to end
Date	August 2026

Accuracy statement Every figure, benchmark, and test result in this document was produced by executing the implementation described, not estimated. Section 13 records known weaknesses, including one measured cryptographic side channel that is not mitigated. NYEDArch is never described as unbreakable.

Contents

If this list appears empty, the table of contents is a live field: select it and press F9 (or choose "Update Field") to populate it with current page numbers.

1. Executive Summary

NYEDArch transforms passive data into an active, self-protecting executable capsule. Rather than storing files inside a container that some separate program must later open, NYEDArch compiles the data together with its own authorization policy, decryption logic, integrity mechanisms and hostile-environment defenses into a single standalone executable. The capsule carries everything it needs. It requires no installed reader, no runtime, and no connection to the system that produced it.

The design assumption is that the capsule will end up somewhere hostile: copied, disassembled, debugged, patched, run repeatedly, and analysed by someone who knows exactly how it works. NYEDArch is built for that assumption rather than against it.

1.1 The core technical claim

Central invariant The payload decryption key is derived from the cryptographic contributions of every enabled authorization factor. A missing or incorrect factor produces a different key, and authenticated decryption fails. There is no boolean that gates access to a key which exists independently of that boolean.

This distinguishes NYEDArch from the common pattern in protected-archive software:

```
// The pattern NYEDArch deliberately does NOT use:
if (authorized) { decrypt_with(EMBEDDED_KEY); } // one patched branch defeats it

// The NYEDArch construction:
key = HKDF( machine_secret || passphrase_key || location_cell || time_window )
// no factor -> no contribution -> wrong key -> AEAD authentication fails
```

The practical consequence is that patching the authorization check does not yield the data. This was verified experimentally rather than argued: the binding validation branch was removed from the runtime source, the capsule was recompiled, and a transplanted package was supplied. Execution passed the neutralised branch and still failed closed one state later, because the key material it needed was never reachable. Section 11.4 records the transcript.

1.2 Why it exists

Conventional data protection is environmental. A file is safe because it sits inside something safe - an encrypted disk, a network behind a firewall, a server enforcing access control, a reader honouring a policy. Every one of those shares a failure mode: the moment the data leaves that environment, the protection is gone. The file is copied to a drive, forwarded, exfiltrated or carried out by someone on their last day, and from that instant it is an ordinary file. This recurs every year at enormous cost, not because the protections were badly built, but because they were never attached to the thing being protected.

NYEDArch starts from a different question: what if the data protected itself, instead of relying on the environment it happens to be sitting in? A capsule is not an executable that enforces a policy on data - that description is accurate and misses the point. It is data that has stopped being passive.

The intended full system extends that into a chain: an unauthorized attempt yields nothing, leaves a trace, notifies the owner within days rather than months, blocks the machine it came from, and can ultimately destroy the specimen - turning an attacker's unlimited patience into a shrinking window. Several of those links cannot exist inside a capsule running alone on a hostile machine, which is why the licensed architecture makes connectivity mandatory rather than optional. The full statement is in docs/PHILOSOPHY.md, along with the legal boundaries that constrain it.

1.3 What was built

Component	Description	Status
Builder client	Rust desktop application: file selection, fingerprint management, protection configuration, sealing, runtime generation, GitHub orchestration, capsule launching	Complete
Generated capsule	Per-package standalone executable containing the sealed payload, authorization policy and extraction logic	Complete
Cryptographic core	Auditable confidentiality boundary: no unsafe code, no I/O, no platform dependency	Complete
Adaptive fingerprinting	Platform-appropriate machine identity across Linux, macOS and Windows	Linux verified
Location	Native provider on all three platforms, browser consent flow as the universal fallback	Linux/Windows tested
Remote build pipeline	GitHub Actions as an untrusted build environment with artifact provenance verification	Verified end to end
Desktop client	Original design language, stepped workflow, drag-and-drop launcher, licence gating, full parity with the command line	Complete
Client hardening	Stripped and link-time optimised; anti-analysis check shared by both clients	Complete

1.4 Verification summary

Measure	Result
Automated tests	142 passing, 0 failing
Compiler warnings	0 across all crates
Unsafe code in cryptographic core	Forbidden at compile time (<code>#![forbid(unsafe_code)]</code>)
End-to-end pipeline	Executed: seal → generate → remote build on GitHub → capsules for all three targets → authorized extraction → byte-identical output
Adversarial review	15 attack questions answered; 6 defects found and fixed
Seal throughput	~172 MiB/s (zstd, release)
Restore throughput	~604 MiB/s (zstd, release)
Known unmitigated weaknesses	5, documented in Section 13

2. Problem Statement and Market Position

2.1 The gap

Data protection today divides into approaches that each break down at the same point — the moment data leaves a controlled environment.

Approach	How it protects	Where it fails
Encrypted archives (ZIP/7z/RAR)	Password-derived symmetric key	A password is portable. Anyone who learns it can open the archive anywhere, on any machine, forever. No machine, place, or time binding.
Full-disk encryption	Protects data at rest on that disk	Irrelevant once a file is copied off the disk. Protects the container, not the content.
Enterprise DRM / IRM	Server-mediated policy enforcement	Requires infrastructure, licensing, connectivity, and a compatible reader on the recipient side. Fails offline and in air-gapped settings.
Cloud access control	Server enforces who may download	Enforcement ends at download. The file is then unprotected.
Self-extracting archives	Convenience packaging	Typically a trivially patched branch. Security theatre rather than a cryptographic boundary.

The recurring weakness is that policy and data are separated. Policy lives in a server, a reader application, or an operating system, while the data travels alone and unprotected once it escapes.

2.2 The NYEDArch proposition

NYEDArch inverts this. Policy travels with the data, inside the same artifact, and is enforced cryptographically rather than procedurally.

- The capsule is self-contained: no server, no reader, no network, no installation.
- Authorization is bound to machine identity, a passphrase, and optionally to place and time.
- Enforcement is cryptographic: failing a condition means the key cannot be derived, not that a check returned false.
- Prototype-I works offline and air-gapped, which server-mediated DRM cannot.

FUTURE — LICENSE SERVER INTEGRATION. The first licensed version makes connectivity mandatory in exchange for revocation, authenticated time, identity, and audit. That is a deliberate trade of the offline property for central control, and it is stated as a trade rather than a pure gain: field and air-gapped deployments lose a capability that Prototype-I has.

2.3 Applicable scenarios

Scenario	Which protections apply	Value delivered
Sending sensitive material to a specific contractor	Machine + passphrase	Forwarding the file to a third party is useless — it will not open on their hardware
On-site-only engineering data	Machine + passphrase + location	Data opens inside the facility and nowhere else
Scheduled release (embargo, results, tender)	Machine + passphrase + time	Recipients hold the capsule in advance; it opens only within the permitted window

Scenario	Which protections apply	Value delivered
One-time credential or key handover	Machine + passphrase + one-shot	Capsule destroys itself after a single verified extraction
Air-gapped or field distribution	All protections	No infrastructure dependency of any kind

2.4 Positioning statement

Honest framing NYEDArch raises the cost, difficulty and expertise required for unauthorized extraction. It is not, and is never claimed to be, unbreakable. Section 12 states plainly what an attacker in possession of a capsule can recover. Any evaluation of this technology should begin there.

3. System Architecture

3.1 Two systems, deliberately separated

NYEDArch consists of two programs with almost no shared code. This separation is a security requirement, not an organisational preference: the capsule runs in hostile territory, so it must not contain capabilities it does not need.

	Builder client	Generated capsule
Runs on	Trusted operator machine	Any machine, assumed hostile
Lifetime	Installed permanently	One artifact per sealed package
Can construct packages	Yes	No — compiled out
Holds GitHub credentials	Yes	Never
Holds private key material	Yes, in OS secure storage	Never
Contains a GUI	Yes	No
Embedded secrets	None in artifacts it emits	Bootstrap key + runtime commitment only

Enforcement is mechanical rather than advisory. Package-construction code sits behind a Cargo feature named "builder" which is off by default; only the client enables it. The shipped capsule is compiled with default features, so the code is absent from the binary. This was verified by inspecting the built artifact (Section 11.6).

3.2 Crate structure

Crate	Lines	Responsibility	In capsule
nyedarch-crypto	973	Key composition, AEAD, KDF, geo and time factors. The confidentiality boundary.	Yes
nyedarch-core	373	Identifiers, policy record, capsule identity, launcher	Yes
nyedarch-package	751	Package format, streaming pipeline, restore	Yes (builder feature off)
nyedarch-fingerprint	396	Adaptive machine fingerprinting, .nyfp records	Yes
nyedarch-platform	116	Location acquisition	Yes
nyedarch-runtime	922	Authorization state machine, hardening	Yes
nyedarch-buildtool	1,034	Sealing, generation, remote build, benchmarks, EULA	No
nyedarch-github	1,012	Build orchestration, provenance, HTTP transport	No
nyedarch-gui	~240	Desktop client interface	No

3.3 The cryptographic core as an audit target

nyedarch-crypto is deliberately the smallest component that must be trusted for confidentiality. It is constrained so that it can be reviewed in isolation:

- `#![forbid(unsafe_code)]` — unsafe code is a compile error, not a convention.
- No file, network or process I/O of any kind.
- No platform-specific code and no operating-system dependency.

- Randomness is supplied by the caller through an injected trait, so the crate makes no OS calls and deterministic test vectors are possible.

```
$ cargo build -p nyedarch-crypto --no-default-features
  Finished `dev` profile [unoptimized + debuginfo] target(s)

# builds with zero platform surface; getrandom is not linked
```

3.4 End-to-end data flow

```
BUILDER (trusted machine)
  files/folders
    -> manifest construction (paths, modes, mtimes, symlinks)
    -> streaming chunker (bounded memory)
    -> compression (zstd, per-chunk)
    -> AEAD sealing (XChaCha20-Poly1305, AAD-bound)
    -> package assembly (header + sealed policy + manifest + chunks)
    -> runtime source generation (per-build diversified)
    -> GitHub Actions build [untrusted environment]
    -> artifact provenance verification
    -> capsule.nyarch

CAPSULE (hostile machine)
  integrity -> binding -> policy bootstrap -> fingerprint -> passphrase
  -> location? -> time? -> key composition -> authenticated decryption
  -> decompression -> restore -> cleanup -> optional self-destruction
```

3.5 Trust boundaries

Boundary	Trusted	Untrusted
Builder machine	Operator, OS keystore, local filesystem	—
GitHub Actions	Nothing	Build environment, runner, returned artifacts
Transport to recipient	Nothing	Any channel; the capsule protects itself
Recipient machine	Nothing	OS, debugger, memory, clock, filesystem, user

GitHub is explicitly untrusted. It never receives plaintext payload data, the passphrase, raw fingerprint data, raw coordinates, or any payload key. The payload is sealed before it leaves the client, so full compromise of the repository or runner does not yield plaintext.

4. Cryptographic Design

4.1 Primitives

NYEDArch invents no cryptographic primitives. Novelty is confined to composition, policy enforcement, runtime architecture and binding — the areas where a new construction is defensible. Every underlying primitive is established and widely reviewed.

Purpose	Primitive	Rationale
Authenticated encryption	XChaCha20-Poly1305	192-bit nonce space makes random nonces safe without a counter; no reliance on AES hardware, so performance is uniform across target machines
Passphrase hardening	Argon2id	Memory-hard; the established modern choice. Resists GPU and ASIC parallel search
Key derivation and expansion	HKDF-SHA512	Standard extract-and-expand with explicit domain separation via info strings
Record authentication	HMAC-SHA512	Portable fingerprint and EULA acceptance records
Digests and commitments	SHA-256	Signal digests, package and source commitments
Compression	zstd (default), DEFLATE	Codec recorded per package so older capsules keep opening

4.2 Domain separation

Every derivation passes a distinct label through HKDF, so identical input keying material produces unrelated outputs in different domains. This is what mechanically enforces the separation rule between key classes.

nyedarch:v1:factor:machine	nyedarch:v1:factor:passphrase
nyedarch:v1:factor:location	nyedarch:v1:factor:time
nyedarch:v1:payload-key	nyedarch:v1:policy-seal
nyedarch:v1:nyfp	nyedarch:v1:artifact-provenance

4.3 Payload key composition

The payload key is the security core of the entire system. Its derivation is reproduced here in full because everything else depends on it.

```

ikm = HKDF(salt, machine_secret, "factor:machine")
      || HKDF(salt, passphrase_key, "factor:passphrase")
      || HKDF(salt, location_cell, "factor:location") // iff enabled
      || HKDF(salt, time_window, "factor:time") // iff enabled

context = "payload-key" || crypto_version || policy_flags
           || package_id || runtime_binding

K_payload = HKDF(salt = package_salt, ikm = ikm, info = context)

payload_aad = context // same bytes bound as AEAD associated data

```

Four properties follow directly from this construction:

1. Each factor is domain-separated before concatenation, so no two factors' bytes can be confused or shifted into one another.
2. Presence must match policy exactly. A contribution supplied for a disabled protection, or missing for an enabled one, is rejected rather than ignored.

3. Binding data enters both the HKDF info and the AEAD associated data, so a payload cannot be transplanted into a different runtime.
4. Machine and passphrase flags are checked explicitly and cannot be cleared, which is how "always on" is enforced cryptographically rather than by UI convention.

Design note on low-entropy factors Location and time are policy factors, not entropy sources. An attacker who knows the policy can enumerate plausible cells and windows. NYEDArch never claims they add meaningful entropy. Their security value is that they must be satisfied in addition to the machine and passphrase protections.

4.4 Complete key hierarchy

The separation rule enforced throughout:

```
PAYLOAD CRYPTO != AUTHORIZATION CRYPTO != GITHUB TRANSPORT CRYPTO
                != ARTIFACT AUTHENTICITY != LOCAL KEYSTORE PROTECTION
```

4.4.1 Payload crypto

Property	K_payload — payload encryption key
Origin	Derived; never stored, never transmitted
Derivation	HKDF-SHA512 over concatenated factor contributions
Persistence	None. Exists only in RAM during a build or an authorized run
Storage location	Nowhere — not in the package, not in the binary, not on GitHub
Lifetime	From composition until the final chunk is processed
Rotation	Per package; a new build means new salt and binding, hence an unrelated key
Zeroization	Zeroizing<[u8;32]>, wiped on drop
Accessible to	Builder at seal time; capsule only after full authorization

4.4.2 Authorization crypto

Property	S_machine — per-fingerprint factor secret
Origin	Freshly minted 32 bytes from the OS CSPRNG at build time, one per trusted fingerprint
Derivation	Not derived — minted. Deliberately independent of the fingerprint value
Purpose	The machine protection's contribution to the payload key
Storage location	Inside the sealed policy record within the package
Lifetime	Build time, then per-run from record unseal until key composition
Zeroization	Wiped with the decrypted record buffer
Accessible to	Builder (mint) and capsule (select on match)

Each trusted fingerprint receives its own independent secret, so extracting one capsule's record does not reveal material belonging to another machine.

FUTURE — LICENSE SERVER INTEGRATION. A fifth contribution, a per-capsule server key share, joins the composition. It is absent from the binary entirely, so extracting a capsule no longer yields every local factor — which directly addresses the S_machine weakness below. What remains: the capsule then cannot open without connectivity, and hardware attestation is still the real fix for machine identity.

Property	K_bootstrap and C_runtime
Origin	Freshly minted 32 bytes each, per build
Purpose	Together derive the subkey that seals the policy record; C_runtime also binds package to runtime
Storage location	Embedded as constants in the generated capsule
Attacker access	Recoverable by anyone who owns the capsule — assumed public to a determined analyst
Accessible to	Capsule; the builder uses them once at seal time

```
K_policy_seal = HKDF(package_salt, K_bootstrap || C_runtime, "policy-seal")
```

4.4.3 GitHub transport and artifact authenticity

Channel	Direction	Purpose	Private half held by
A	Client → GitHub	Authenticity of build inputs	Client (OS secure storage)
B	GitHub → Client	Artifact provenance signatures	Runner (as a repository secret)

Why two independent keypairs Reuse across directions is prohibited. A single keypair would allow a compromised runner to mint build inputs the client accepts, or to forge artifacts. Neither channel key participates in any payload derivation, and their domain labels are disjoint from all payload labels — so a transport key can never become a payload decryption key.

4.4.4 Component access matrix

Key	Builder	Capsule	Runner	Repository
K_payload	derives at seal	derives after auth	never	never
S_machine	mints	selects on match	never	never
K_bootstrap	mints, uses once	embedded	never	never
C_runtime	mints	embedded	never	header only
Channel A private	holds	never	never	never
Channel B signing	never	never	uses	as secret
Channel B verifying	holds	never	never	never
Keystore master	OS-held	never	never	never

The capsule column is the security-relevant one. A capsule in hostile hands carries K_bootstrap and C_runtime and nothing else secret.

4.5 The fingerprint bootstrap problem

A capsule must keep its trusted-fingerprint list confidential, but cannot require the fingerprint in order to decrypt the fingerprint list. That circularity is a trap this design had to solve explicitly.

Question	Answer
What is public?	K_bootstrap, C_runtime, package salt and id, policy flags, KDF parameters, and all code
What is protected?	The policy record: trusted fingerprint digests, their S_machine secrets, and labels
How is it unlocked?	K_policy_seal = HKDF(salt, K_bootstrap C_runtime, label) — no fingerprint required, so no circularity

Question	Answer
What does an attacker gain by unsealing it?	The hashed fingerprint ids and the S_machine secrets. This is a real, deliberate exposure.
Why is that survivable?	The fingerprint ids are salted digests revealing no raw hardware identifiers, and S_machine alone is useless — the payload key also requires the Argon2id passphrase contribution

The resolution is that a fingerprint never becomes a key; it selects a high-entropy secret. A non-matching fingerprint selects nothing, and nothing cannot be composed into a key.

Stated limitation Because S_machine is extractable from a capsule an attacker possesses, the machine protection is not a standalone confidentiality boundary. It constrains where a capsule is usable and forces per-capsule reverse engineering. Closing this gap requires hardware-attested identity (TPM sealing or Secure Enclave) so the secret never exists in extractable form. That is the designated strengthening path and is not implemented in this prototype.

5. Authorization Model — The Four Protections

5.1 Combination semantics

Protection	Status	Combination
Machine	Mandatory — cannot be disabled	AND
Passphrase	Mandatory — cannot be disabled	AND
Location	Optional	AND when enabled
Time	Optional	AND when enabled

Enabled protections combine with AND: every one must succeed. Within the machine protection, multiple trusted fingerprints combine with OR — any single match authorizes. The creator's own machine is always included, since the operator is by definition trusted.

```
authorized = (fp == A || fp == B || fp == C)  // OR across fingerprints
              AND passphrase
              AND location      // iff enabled
              AND time          // iff enabled
```

In the interface these are independent checkboxes, never radio buttons, because they are composable protections rather than mutually exclusive modes. The two mandatory protections render as locked and cannot be switched off.

5.2 Machine protection

Contribution: the 32-byte `S_machine` secret selected by a matching fingerprint. Failure mode: no match selects no secret, so no key can be composed.

5.3 Passphrase protection

Contribution: Argon2id output over the passphrase with a per-package random salt. Parameters are versioned and stored in the package header so they can evolve without breaking existing capsules.

Profile	Memory	Iterations	Parallelism	Measured time
Prototype default	64 MiB	2	1	~96–223 ms
Interactive (recommended)	256 MiB	3	1	~684 ms – 1.36 s

No verifier is stored anywhere. There is nothing in the capsule against which a candidate passphrase can be checked except attempting the full derivation and authenticated decryption, which is what makes the memory-hard cost meaningful. The passphrase never appears in logs, configuration, crash output, package metadata, or GitHub.

5.4 Location protection

Raw satellite coordinates are unstable, so they cannot be compared directly. NYEDArch quantizes a reading into a deterministic grid cell scaled by the cosine of latitude, and that cell identifier becomes the factor contribution.

5.4.1 Accuracy rejection

A reading whose reported accuracy is worse than the configured tolerance is rejected outright rather than accepted optimistically:

```
configured tolerance = 100 m
reported accuracy    = 2000 m
-> reject: the device cannot prove it is inside the authorized region
```

5.4.2 No manual entry, on either side

There is deliberately no field anywhere in NYEDArch for typing coordinates. At build time the client refuses to seal a package with location protection when no provider is present, rather than accepting typed input. At run time the capsule fails closed when its provider is unavailable.

```
$ nyedarch-buildtool seal ./docs ./out "pw" --location 100

error: the location protection was requested but no trustworthy location
       provider is available on this machine.

       NYEDArch does not offer manual coordinate entry as a fallback,
       because a typed-in location would make the protection meaningless.
```

Platform providers: GeoClue2 on Linux, CoreLocation on macOS, Windows.Devices.Geolocation on Windows. Where a provider is not implemented the code returns nothing rather than a fabricated reading — a fabricated location would be precisely the silent security downgrade the design forbids.

5.5 Time protection

The operator configures a recurring daily schedule: one or more times of day, a symmetric tolerance window, and a timezone offset. The capsule reads its own clock and computes the window itself; the user is never asked to type the current time.

5.5.1 Day invariance — a design defect found and corrected

Defect found during implementation The first implementation derived the window identifier from the absolute instant, which included the day. A schedule of "every day at 14:00" would therefore have produced a key that worked only on the day the capsule was built — silently converting a recurring policy into a single-day one. This was caught, corrected to be day-invariant, and locked with a regression test that opens a capsule a full year after sealing.

```
window_id = slot_minutes || tolerance_minutes || tz_offset_minutes
// deliberately day-invariant: a daily schedule must recur
```

The consequence is stated plainly in the documentation and EULA: the time protection constrains when within a day a capsule opens, not which day. It is a recurring-window policy control, not an expiry mechanism.

5.5.2 Clock trust

Stated limitation The prototype reads the local system clock, which whoever controls the machine can change. The time protection is therefore a policy control rather than a tamper-proof expiry. The time source sits behind a TimeSource interface so an authenticated time provider can replace it without redesign.

FUTURE — LICENSE SERVER INTEGRATION. The server supplies authenticated time in its authorization challenge; the time protection is then evaluated against server time rather than the local clock, and excessive skew fails closed. What remains: network delay bounds accuracy, so the tolerance window must exceed plausible round-trip jitter or legitimate users are denied.

6. Adaptive Machine Fingerprinting

6.1 Why a fixed schema was rejected

A naive design collects a fixed set of identifiers — CPU serial, motherboard serial, MAC address. This fails in practice because platforms expose fundamentally different information, and privileges differ between machines. NYEDArch instead discovers whatever trustworthy, stable signals are actually available and records what it found.

Strength	Meaning	Examples
Strong	Firmware, hardware or TPM-backed	DMI product UUID, board serial, TPM presence, macOS IOPlatformUUID, Windows csproduct UUID
Medium	Operating-system install identity	/etc/machine-id, Windows MachineGuid
Weak	Present but not machine-unique	CPU model, hostname

6.2 Privacy: digests, never raw identifiers

Raw values are never stored or displayed. Each signal is reduced to a salted digest:

```
digest = SHA-256( domain || "|sig|" || signal_name || "|" || raw_value )
```

The fingerprint identifier is then derived from all signals at or above a strength threshold: Medium when any Medium-or-better signal exists, otherwise Weak. Signals are sorted by name first, so the result is deterministic.

6.3 Degradation with disclosure

A machine lacking strong signals still produces a usable fingerprint, but the record retains the full inventory of what was available and at what strength. A weaker capture is therefore visible rather than silently substituted for a strong one. Observed on the reference machine, where DMI values were not readable:

```
[buildtool] creator fingerprint: 0cefa0f7a072684b
(best strength Medium, 3 signals)
```

6.4 Machine-state semantics

The fingerprint reflects machine state at capture time. Motherboard replacement, firmware changes, or operating-system reinstallation may legitimately change it, and capsules will then deny that machine. This is intended behaviour rather than a defect, and Prototype-I offers no fallback — a silent one would defeat the protection.

FUTURE — LICENSE SERVER INTEGRATION. A failsafe path restores access after a hardware change: valid licence, MFA code, and out-of-band approval — by the account owner for individuals, or by a team admin with optional additional reviewers for organisations. It substitutes for the machine protection only; the passphrase, location, and time protections still apply. What remains: the failsafe is by construction the weakest link in the machine protection, and a complete account compromise covering password, MFA, and email defeats it.

6.5 Portable .nyfp records

Trusted machines are exchanged as authenticated .nyfp records — versioned, non-human-readable, and covered by an HMAC-SHA512 tag:

```
[u16 version][u32 body_len][body][64-byte MAC]
```

Labels and tags are metadata rather than authorization secrets, but they sit inside the authenticated region. Editing a label from "HR" to "Administrator" invalidates the record. Both tampering and wrong-key cases are covered by tests.

7. Package Format and Streaming Pipeline

7.1 Structure

```
magic "NYARCH" | format version u16
header_len u32 | header (plaintext, versioned, carries no secrets)
policy_len u32 | sealed policy record <- AEAD sealed
manifest_len u32 | sealed manifest <- AEAD sealed
chunk_count u32
[ chunk_len u32 | sealed chunk ] x N <- AEAD sealed, ordered
```

The header is plaintext and deliberately carries no secrets: crypto version, package id, runtime binding, policy flags, Argon2 parameters, package salt, compression codec and chunk size. Framing lengths leak nothing about content. Every secret-bearing section is sealed.

7.2 Associated-data binding

Each chunk's associated data binds the full header context, a section tag, the chunk index and a terminal flag:

```
aad = context || section_tag || chunk_index || is_last
```

Consequently chunk reordering, truncation, section confusion between manifest and payload, and header edits all fail authentication rather than producing wrong output.

7.3 Bounded-memory streaming

Files stream into fixed-size chunks; only one chunk of plaintext exists at any moment. Peak memory is therefore independent of payload size, which is what allows a capsule to handle datasets far larger than available RAM. On restore, decrypted chunks are routed directly into destination files in manifest order — no intermediate archive is ever materialised.

7.4 Manifest fidelity

The versioned manifest preserves relative paths, directory hierarchy, Unix mode bits, modification times, file sizes and symlink targets. On restore, directories are created first in manifest order, files are streamed, and symlinks are created last because their targets may reference restored files.

Path traversal is rejected at validation: absolute paths, empty components and "." components are refused, so a malicious package cannot write outside the extraction root.

7.5 Compression and format evolution

The compression codec is recorded per package, which is what allows the algorithm to change without breaking existing capsules. zstd is the default for new packages; DEFLATE remains fully supported. An unrecognised codec fails closed rather than guessing.

8. Remote Build Pipeline

Capsules are compiled by GitHub Actions. This is the intended flow rather than an optional add-on: the client orchestrates repository lifecycle, sealed secrets, workflow and source push, dispatch, run polling, artifact retrieval and provenance verification as required stages.

8.1 Data minimization

The build environment receives the generated runtime source and the already-sealed package. It never receives plaintext files, the plaintext archive, the passphrase, raw fingerprint data, raw coordinates, or any payload key. Because the payload is sealed before it leaves the client, full compromise of the repository or runner does not yield plaintext.

8.2 Transport and credential handling

The client speaks to the GitHub REST and Git Data APIs over the system curl binary, which ships with Windows 10+, macOS and mainstream Linux. This avoids embedding a large asynchronous TLS stack in a security-sensitive client.

Credential handling Command-line arguments are world-readable through the process list on most systems, so a token in argv would leak to any local user. The Authorization header is therefore written to curl's stdin as a config stream and never appears in argv or in the child environment. This is asserted by a dedicated unit test.

8.3 Repository secrets

GitHub requires repository secrets to be encrypted client-side with the repository's Curve25519 public key using a libsodium sealed box. The orchestrator enforces ordering — the repository public key is fetched before any secret is sealed or written — and this ordering is asserted by test. Where a real sealer is unavailable the client refuses to transmit the secret at all rather than sending it unprotected.

Secrets are never committed to Git, never written to repository files, and never echoed by the workflow. Repository visibility changes are refused while a build is running.

8.4 Artifact provenance

A matching hash proves only that bytes match bytes. It does not establish that an artifact is the expected output of a specific build. The client therefore commits to a build manifest before dispatch and verifies the returned artifact against that commitment.

```
signed_input = "artifact-provenance" || crypto_version
              || build_id || target
              || package_commitment || runtime_commitment
              || source_commitment || artifact_digest
```

Because the signed input is build-specific, a signature captured from one build cannot be replayed onto another. Build id, target, digest and signature are each checked, and any mismatch fails closed. GitHub is never trusted merely because it returned an artifact.

8.4 The capsule source is pushed encrypted

The build environment never receives the generated capsule source in the clear. That source is not neutral: it carries the sealed package, the per-build bootstrap key and the runtime commitment. Everything except a small unlocker is packed into one archive, encrypted with XChaCha20-Poly1305 under a fresh random key, and pushed as ciphertext. The key is stored as a GitHub Actions repository secret, sealed to the repository's own public key, so only a runner can read it.

```

repository:                               runner (build time):
runtime/capsule.sealed ->                build the unlocker
runtime/unlock/**      ->                decrypt with $NYEDARCH_SOURCE_KEY
.github/workflows/...   build the capsule
                        delete the decrypted source

```

The push dropped from 39 files to 3. The unlocker is committed in the clear deliberately: it holds no secret, only the ability to open an archive given a key it does not have. It refuses path traversal and treats any authentication failure as fatal, so a modified archive cannot produce a subtly wrong source tree. The workflow deletes the decrypted source in an `if: always()` step, so plaintext does not outlive the build even when the build fails.

Encryption does not branch on visibility. Both clients can switch a build repository between public and private at any time - `nyedarch visibility <owner> <repo> public|private`, or a button beside the toggle in the desktop client - and the change is refused while a build is running, checked against GitHub rather than local state. A private repository is not a substitute for encryption: it still exposes its contents to collaborators, to anyone later granted read access, and to every clone.

Verified on a genuinely public repository. A reader sees the workflow, a README, the unlocker, and a 179,404-byte blob containing no package magic and no readable source - a nonce followed by ciphertext. All three targets still built.

What this does not protect against The repository owner. Whoever controls the repository controls its workflows, and anything a workflow can decrypt, a modified workflow can print. This moves the exposure from "public by default" to "requires control of the build environment", which is the honest description and not stronger than that.

Stale files are pruned before each push. A repository is reused across builds, so without that a repository used before this change would have kept a full plaintext capsule source indefinitely. After a build the repository contains only the workflow, the encrypted blob and the unlocker.

8.4a Verified against live GitHub

The pipeline has been run end to end against a real account rather than a mock: private repository created, secrets sealed with a libsodium sealed box and stored, workflow and generated capsule source pushed, build dispatched, and capsules produced for every target.

Target	Result	Artifact
x86_64-unknown-linux-gnu	success	339,771 bytes
x86_64-pc-windows-msvc	success	251,280 bytes
aarch64-apple-darwin	success	287,454 bytes

Two defects only a live run could find The macOS target did not compile: `forbid(unsafe\_code)` made every Objective-C call a hard error, and a deprecated CoreLocation function was called without its receiver. Separately, every build after the first into the same repository failed, because the contents API rejects an update that omits the file's current blob SHA - and one repository holding many builds is the intended model, so update-in-place is the normal path. Both are fixed and re-verified.

8.5 Capsule identity: the .nyarch extension

Compiled capsules carry a single extension on every operating system. A capsule is one artifact concept, so it has one name everywhere rather than .exe on Windows and nothing on Unix.

Platform	Terminal execution	Reason
Linux	<code>./capsule.nyarch works</code>	Kernel dispatches on file content (ELF header)
macOS	<code>./capsule.nyarch works</code>	Kernel dispatches on file content (Mach-O header)

Platform	Terminal execution	Reason
Windows	Does not work unmodified	The shell resolves executables via PATHEXT, which does not know this extension

Windows behaviour, stated honestly On Windows the capsule is a valid PE, and CreateProcess runs it regardless of extension — which is why the NYEDArch client's launcher works there. Terminal use requires adding .NYARCH to PATHEXT. Double-click needs a file association and is not assumed to work on any platform. This is an accepted, documented trade-off of a single cross-platform identity, not an oversight.

The workflow stages the binary as nyedarch-<run_id>-<target>.nyarch and sets the executable bit. Windows runners have no execute bit, so that step is a deliberate no-op there. Because artifact archives do not reliably preserve permissions, the client restores the executable bit when launching a capsule.

8.6 Drag-and-drop launching

A finished capsule can be dragged onto the client, or launched from the command line, and runs as a fully independent process.

- The launched capsule performs its own authorization. The client grants it nothing, passes it no key material, and cannot make it open.
- It is spawned with its own stdio, so the client is not in its trust path and never observes the passphrase.
- Non-capsule files are refused with an explanation rather than silently ignored.
- A stripped executable bit is repaired before launch, avoiding a confusing permission-denied failure.

9. The Client Interface

The desktop client is built in Rust with egui, so every part of the interface is drawn by code the project controls and no security-relevant logic sits behind a web view. The visual language is independently designed: a deep obsidian canvas, one warm accent reserved for action, a cool accent reserved for active protection, and components painted directly rather than assembled from stock widgets.

9.1 The idea the interface is built on

The visual language is derived from the product and from the name. NYEDArch is Not Your Everyday Archive: an archive that stops being passive storage and becomes a sealed vessel carrying its own defences into territory nobody controls. Two ideas follow from that.

The canvas is a near-black void with a cold blue cast, because a capsule's destination is unknown and untrusted. Content floats on it in translucent panes rather than lying on a surface, because the capsule is an object in transit.

The identity mark is an aperture: an iris of interlocking blades, like a vault door. It is shut by default and opens only when authorization succeeds, and every protection engaged tightens it. That is the entire product expressed as one shape, and it is reused at every scale - as the application mark, as the build indicator, and as the drop target - so the same idea is restated wherever the user looks.

9.2 Colour carries meaning

Role	Used for	Never used for
Cyan	Live systems and the primary action path	Decoration
Violet	The capsule itself and cryptographic material	General emphasis
Mint	A protection that is engaged	Ordinary success messaging
Coral	Refusal, or an irreversible consequence	Ordinary warnings

9.3 Structure

A menu bar carries the actions that do not belong inside the stepped flow - creating a new capsule, opening one to run, importing and exporting trusted machines, the licence agreement. A navigation rail carries the six stages, with a sliding active indicator and completion marks. A posture meter pinned to the rail shows how many protections are engaged and animates whenever that changes, so enabling one has a visible consequence somewhere other than the control just touched.

9.4 The interface as security communication

Two decisions follow from the security model rather than from taste:

- The mandatory protections render as locked switches, visually distinct from the ones that can be toggled. They are not a user choice, so the control must not imply they are.
- Consequences appear at the point of decision. Choosing a public repository turns its description coral and states what it exposes; enabling one-shot raises a notice that deletion cannot be guaranteed on SSDs.

9.4a The command line

The command-line interface is described declaratively, so help text is generated from the same source as the parser rather than maintained beside it. A help text that drifts from the parser is worse than none: it tells the operator something untrue about a security tool.

Help explains consequences rather than syntax. Reading `--one-shot` tells you deletion is best effort; reading `--location` tells you a vague fix is refused and that an IP lookup is never used; reading `--hardware required` tells you the build is refused rather than downgraded. A test asserts those phrases are present.

Values are validated against allowed sets, so a typo cannot select a looser policy. `--tag-mode` and `--hardware` each have a safe and a permissive value, and silently guessing wrong would either trust more machines than asked or drop a protection. Running with no arguments shows help rather than doing something, and `--help` and `--version` work without touching the keystore, the licence record or the fingerprint engine.

9.5 Parity with the command line

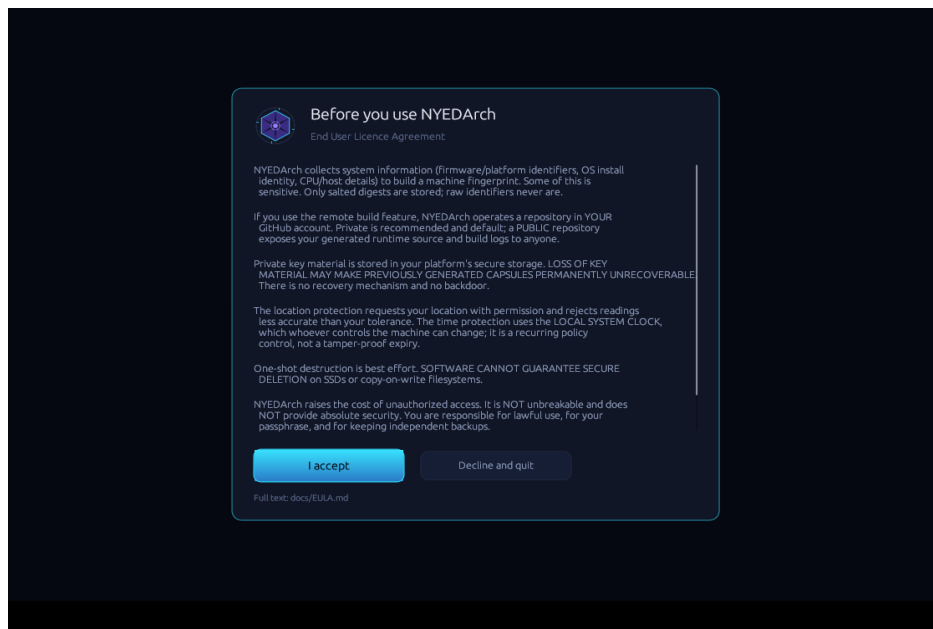
Both clients call the same library. There is deliberately no separate implementation behind the desktop interface, because a second code path could drift from the real one and an interface would end up reporting progress for work that did not happen.

Capability	Command line	Desktop
Licence gating before any use	yes	yes - blocks the whole window
Sealing pipeline	shared library	shared library
Machine and passphrase protections	yes	yes
Location, native then browser	yes	yes
Time window	yes	yes
One-shot capsules	yes	yes
Trusted machine import and export	yes	yes
Machine search, tags, ANY/ALL selection	yes	yes
Compression modes	yes	yes
Build cancellation	yes	yes
Creator diagnostics	yes	yes
Remote GitHub build	yes	yes
Capsule launching	yes	yes, plus drag and drop
Client anti-analysis check	yes	yes

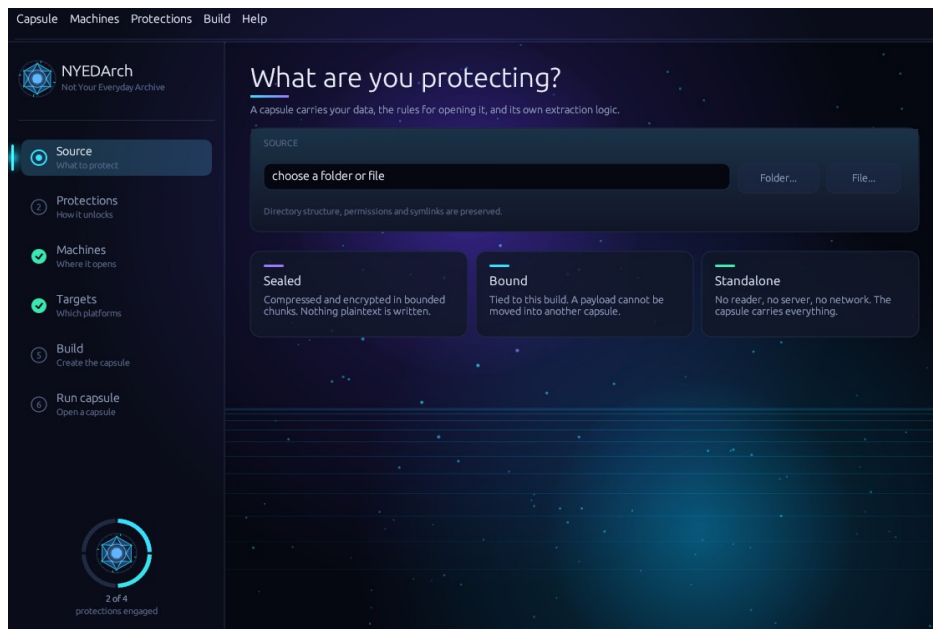
Progress in the desktop client advances only when the pipeline reports a stage beginning. Sealing runs on a worker thread because Argon2id is deliberately slow and would otherwise freeze the window.

9.6 Screens

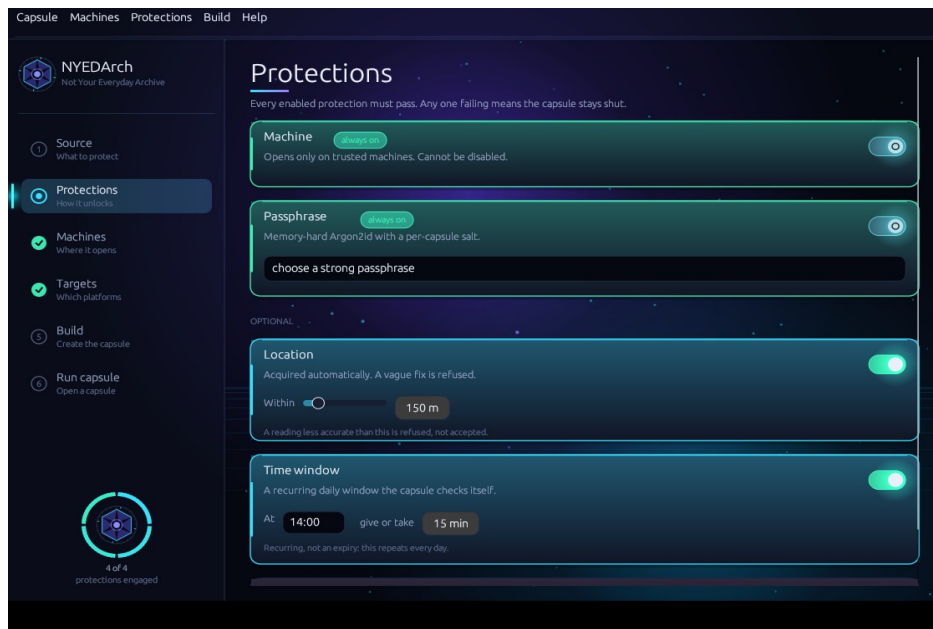
Captured from the running client.



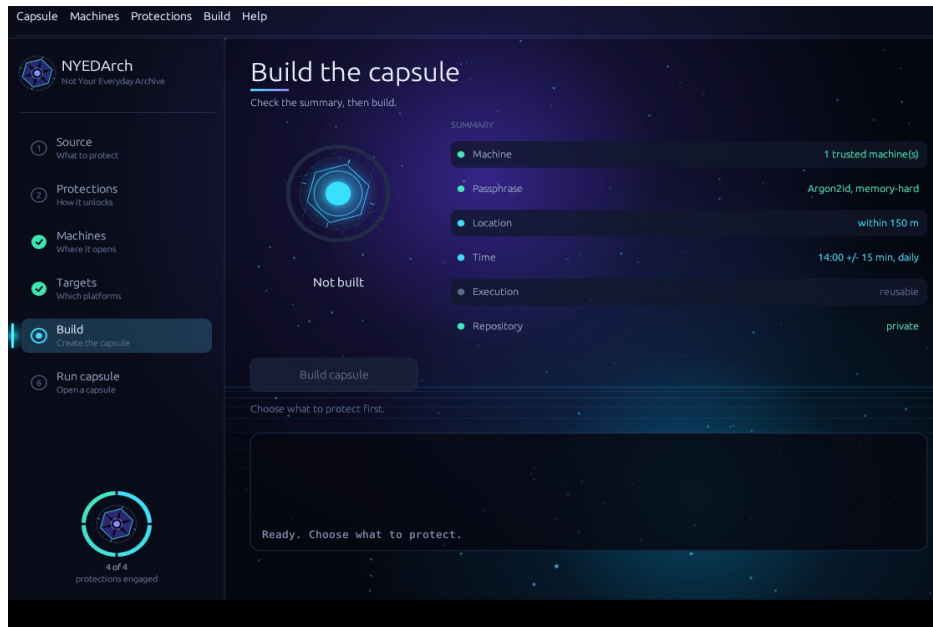
The licence gate. Nothing else in the application is reachable until this is accepted.



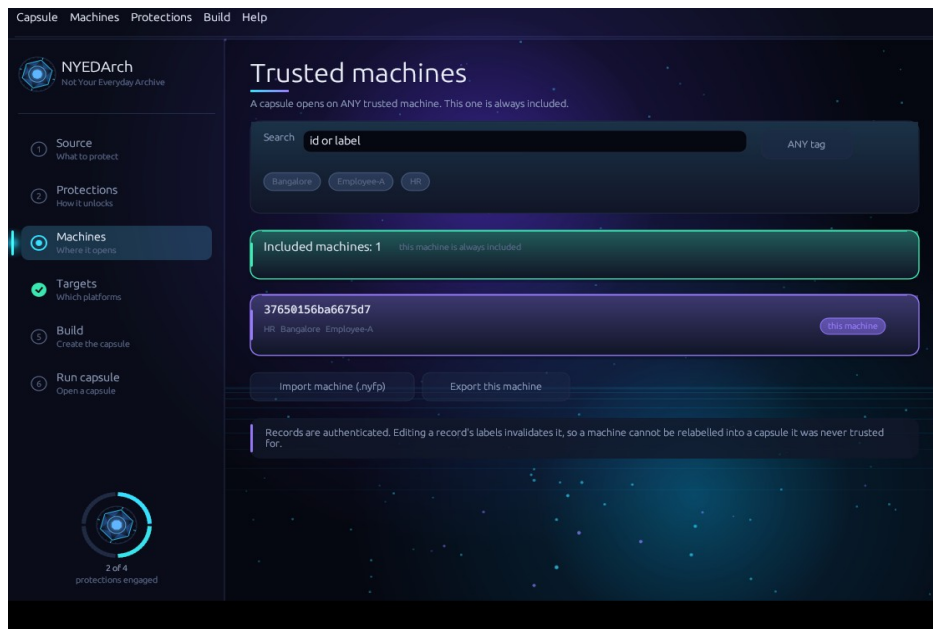
Source - choosing what to protect, with native folder and file pickers.



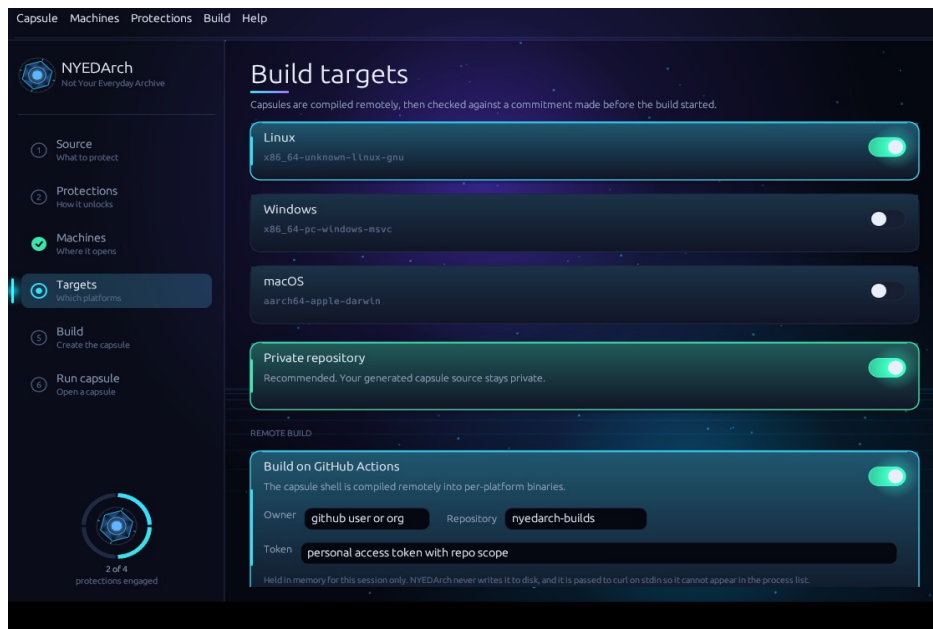
Protections - mandatory protections shown as locked switches; optional ones expand to reveal their settings.



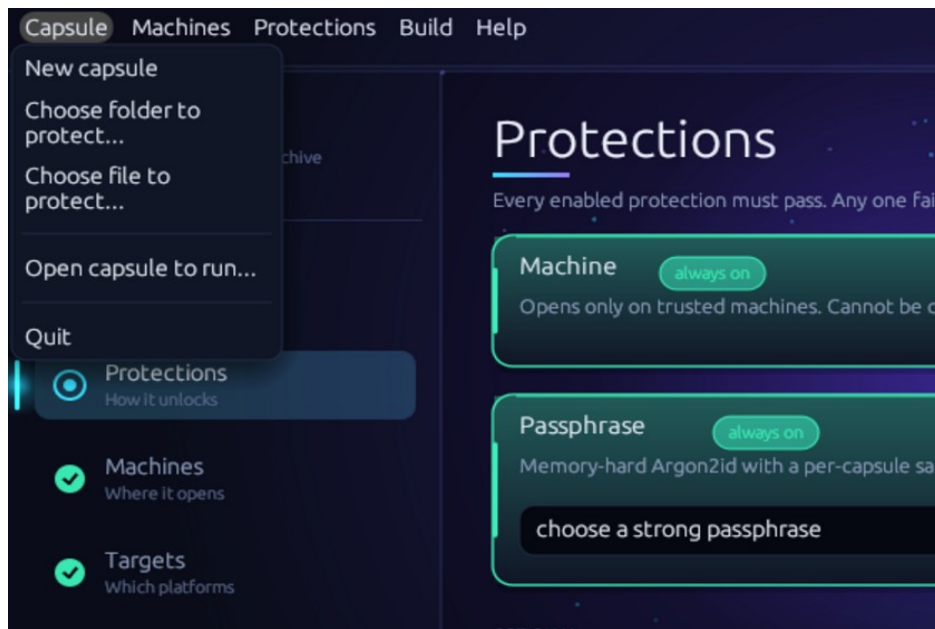
Build - the aperture and progress arc beside the capsule summary.



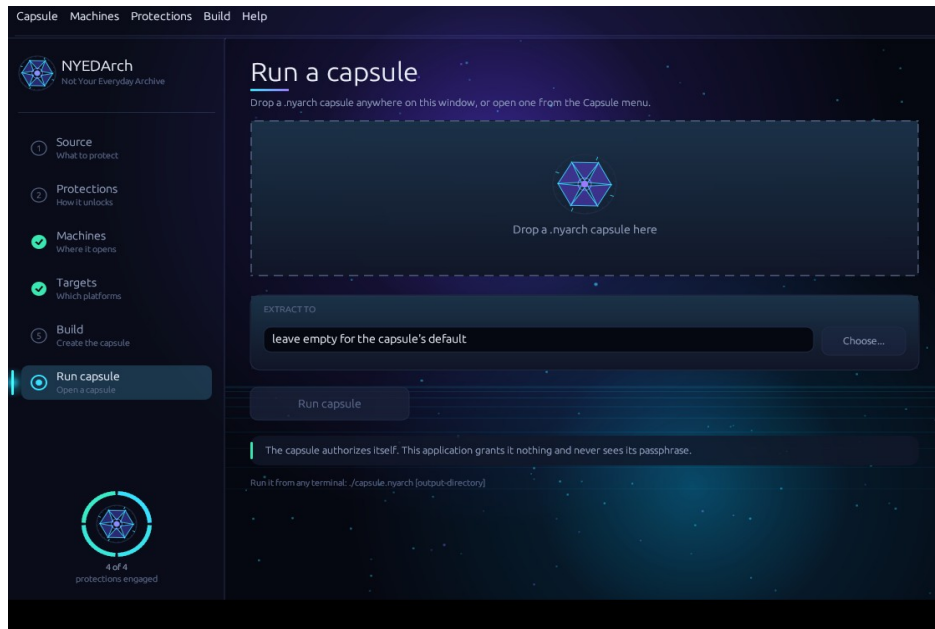
Machines - the shared registry with search, tag filters, ANY/ALL semantics, and the count of machines the capsule will trust.



Targets - build platforms, repository visibility, and the GitHub credentials used for the remote build.



Menus - actions that live outside the stepped flow: new capsule, open a capsule, import and export machines, and the licence.



Run - the drag-and-drop target for launching a finished capsule as an independent process.

10. Test Suite — Complete Register

162 automated tests, all passing, with zero compiler warnings. Every security-relevant negative test asserts two things: that authorization was denied, and that no plaintext reached the output directory.

Suite	Tests	Focus
nyedarch-crypto (unit)	4	AEAD, geo rejection, policy seal, time windows
nyedarch-crypto (invariant)	6	The central security invariant under attack
nyedarch-core	6	Capsule identity, extension handling, launcher
nyedarch-fingerprint	2	Determinism, authenticated records
nyedarch-package	18	Round-trip fidelity, codec compatibility, 13 executed attacks incl. an exhaustive byte sweep
nyedarch-platform	17	Native providers, browser consent flow, hardware capability and policy
nyedarch-runtime (unit)	4	Hardening accumulator
nyedarch-runtime (authorization)	17	Full state machine, all four protections, constant-shape denial
nyedarch-runtime (destruction)	17	Overwrite, unlink, verify; delegation, inode-level scrub evidence, permission, handle, copy and tree cases
nyedarch-github	12	Orchestration, provenance, credentials, sealed-box secrets, stale-file pruning
nyedarch-buildtool	23	Protection parsing, anti-analysis, key storage, encrypted packing, machine registry
nyedarch-installer	4	macOS bundle layout and Info.plist correctness
Total	162	0 failures, 0 warnings

10.1 Central invariant tests

These are the tests that matter most: they attack the payload key directly.

Test	What it proves	Result
authorized_open_succeeds_and_binds_runtime	Correct factors open the payload; runtime binding is enforced	PASS
wrong_fingerprint_secret_denies_payload	A wrong machine secret yields a different key; decryption fails	PASS
wrong_passphrase_denies_payload	A wrong passphrase yields a different key; decryption fails	PASS
missing_enabled_factor_fails_closed	Omitting an enabled factor is rejected, not silently ignored	PASS
attacker_cannot_downgrade_policy_to_drop_a_factor	Clearing a policy flag does not remove a factor requirement	PASS
ciphertext_tamper_is_detected	Modified ciphertext fails authentication	PASS

10.2 Authorization state machine tests

Test	What it proves	Result
authorized_run_extracts_payload	Full pipeline succeeds with correct factors	PASS
wrong_passphrase_is_denied_and_leaks_nothing	Denial plus empty output directory	PASS
untrusted_machine_is_denied	A package trusting another machine denies this one	PASS
unavailable_passphrase_fails_closed	No passphrase source means no authorization	PASS
transplanted_package_is_denied	Runtime A with package B cannot open	PASS
tampered_payload_fails_authentication	A flipped ciphertext byte is detected	PASS
corrupted_header_fails_closed	A broken magic value fails cleanly	PASS
location_ward_accepts_same_region	Correct region authorizes	PASS
location_ward_denies_different_region	A different city is denied	PASS
location_ward_rejects_poor_accuracy	Right place, insufficient accuracy, denied	PASS
location_ward_fails_closed_without_provider	No provider means denial, never a bypass	PASS
time_ward_accepts_inside_window	In-window execution authorizes	PASS
time_ward_denies_outside_window	Out-of-window execution denied	PASS
time_ward_is_day_invariant	A daily schedule still opens a year later	PASS
all_wards_together_require_every_factor	AND semantics: breaking any one protection denies	PASS

10.3 Supporting tests

Test	What it proves	Result
capture_is_deterministic	Same machine state reproduces the same fingerprint	PASS
nyfp_roundtrip_and_tamper	Edited or wrong-key records are rejected	PASS
seal_then_restore_reproduces_tree	Byte-identical restoration including multi-chunk and empty files	PASS
both_codecs_roundtrip_and_stay_readable	zstd and DEFLATE packages both open	PASS
unknown_codec_fails_closed	An unrecognised codec is refused, not guessed	PASS
credential_goes_to_the_config_stream_no_t_argv	Tokens never reach the process list	PASS
full_build_sequence_is_ordered	Repository key fetched before any secret is written	PASS
visibility_change_refused_during_build	Visibility cannot change mid-build	PASS
workflow_yaml_has_dispatch_and_targets	.nyarch naming, chmod, no secret echoing	PASS
accepts_expected_artifact	Valid provenance is accepted	PASS
rejects_wrong_build_target_or_digest	Mismatched build, target or digest rejected	PASS
rejects_unsigned_or_replayed_signature	Cross-build signature replay rejected	PASS
launch_restores_executable_bit_and_spaw	Launcher repairs permissions and starts the process	PASS

Test	What it proves	Result
ns		
rejects_non_capsule_files	Non-capsule files refused	PASS
extension_is_stable_and_recognized	.nyarch recognised case-insensitively	PASS
rejects_bad_input_rather_than_disabling_a_ward	A mistyped flag errors instead of silently disabling protection	PASS

11. Experimental Results

11.1 End-to-end pipeline

```
$ nyedarch-buildtool seal ./secret_docs ./capsule "passphrase"
[buildtool] creator fingerprint: 0cefa0f7a072684b (Medium, 3 signals)
[buildtool] sealing 5 entries
[buildtool] package sealed: 20563 bytes

-- SECURITY REVIEW -----
MACHINE AUTHORIZATION    mandatory - 1 trusted fingerprint(s)
PASSPHRASE                mandatory - Argon2id m=64MiB t=2 p=1
GEOLOCATION                disabled
TIME                     disabled
EXECUTION                 reusable
-----

$ sh ./stage-capsule.sh
capsule: nyedarch-40d9f326c43dfdf1.nyarch (725,832 bytes, -rwxr-xr-x)

$ NYEDARCH_PASSPHRASE="passphrase" ./nyedarch-*.nyarch ./out
NYEDArch: extraction complete - 3 files, 2 dirs, 20056 bytes -> ./out

$ diff -r ./secret_docs ./out/secret_docs -> IDENTICAL
```

11.2 Denial behaviour

```
$ NYEDARCH_PASSPHRASE="wrong" ./nyedarch-*.nyarch ./out_bad
Authorization failed.
exit=1 output directory: not created
```

A single generic message is emitted regardless of which protection failed, so normal execution does not act as an authorization oracle. Detailed state is available only through the diagnostic trace, which is explicitly documented as providing no additional privilege.

11.3 Time protection, live

```
# Capsule sealed for the current window:
[state] TimeAcquisition -> KeyDerivation -> ... -> Success
NYEDArch: extraction complete - 3 files, 2 dirs, 20056 bytes

# Identical capsule sealed for a window three hours away:
[state] PassphraseAcquisition -> TimeAcquisition -> FailClosed
Authorization failed.          leaked: nothing
```

11.4 The branch-patching experiment

This is the experiment that substantiates the central claim. Rather than asserting that authorization is cryptographic rather than boolean, the binding validation comparison was physically removed from the runtime source, the capsule was recompiled, and a package from a different build was transplanted into it.

```
// attacker patch simulation applied to the runtime source:
// -   if diff != 0 { fail!(); }
// +   let _ = diff;    // comparison neutralised

Result with the check REMOVED:
[state] IntegrityCheck
[state] BindingValidation    <- patched branch passed
[state] BootstrapPolicy
```

```
[state] FailClosed      <- still denied
Authorization failed.
leaked: nothing
```

Interpretation The patched branch was passed. Extraction still failed, because the policy-seal subkey derives from the runtime's own embedded commitment rather than from any value the attacker controls. The attacker gained exactly one state transition and no plaintext. This is what "cryptographic rather than boolean-gated" means in practice.

11.5 Transplantation resistance

Attack	Outcome	Mechanism
Runtime A + package B	Denied at BindingValidation	Header commitment differs from the runtime's own
Same, with the check patched out	Denied at BootstrapPolicy	Subkey derives from the runtime's own commitment
Copied sealed payload into another capsule	AEAD authentication failure	Binding is in both HKDF info and AAD
Modified manifest	AEAD authentication failure	Manifest is a sealed, tagged chunk
Chunk reorder or truncation	AEAD authentication failure	AAD binds index and terminal flag
Artifact substitution from the build environment	Provenance rejection	Signature covers build id, target and all commitments

11.6 Capability isolation verification

```
$ cargo tree -p nyedarch-package -f "{p} {f}" # in the real capsule build
nyedarch-package v0.0.1 default

$ strings ./nyedarch-*.nyarch | grep -c "seal_package|nyedarch_package::build"
0 -> builder capability absent from the shipped capsule
```

11.7 EULA gating

Case	Behaviour	Result
Decline	Application exits without operating	Blocked
Accept	Authenticated acceptance record written	Recorded
Subsequent runs	Not re-prompted	Persisted
Record tampered (one byte flipped)	Record rejected, user re-prompted	Detected

12. Performance Benchmarks

All figures were measured by executing the release build on the reference machine. Nothing here is estimated. The reference machine is a single-core containerised Linux environment, so absolute numbers are conservative; relative comparisons are the meaningful part.

12.1 Core operations

Operation	Measured	Note
Fingerprint capture	76–82 μ s	3 signals, Medium strength
Argon2id 64 MiB, t=2	96–223 ms	Prototype default
Argon2id 256 MiB, t=3	684 ms – 1.36 s	Recommended interactive profile
Capsule authorized run (small payload)	~101 ms	Dominated by Argon2id
Capsule denial, early protection	~2.0 ms	Fails before Argon2id — see 12.2
Capsule denial, passphrase protection	~95–105 ms	Argon2id cost incurred

12.2 Throughput by codec

Codec	Payload	Seal	Seal MiB/s	Restore	Restore MiB/s	Ratio
DEFLATE	8 MiB	0.05 s	151.2	0.01 s	601.0	20.7x
DEFLATE	32 MiB	0.21 s	153.3	0.05 s	587.2	20.7x
zstd	8 MiB	0.05 s	171.8	0.02 s	427.6	23.7x
zstd	32 MiB	0.19 s	172.0	0.05 s	603.7	23.7x

zstd is the default for new packages: better ratio and better throughput on both paths.

12.3 A performance defect the benchmarks caught

Defect found by measurement The first zstd implementation bounded decompression with a fixed 64 MiB cap, which allocated 64 MiB for every 1 MiB chunk. Restore throughput collapsed to roughly 22 MiB/s — slower than DEFLATE, which is precisely what made the defect visible. It was corrected by recording the exact uncompressed length inside the AEAD-sealed chunk, so allocation happens once and a forged length cannot force a large allocation.

Stage	Restore MiB/s	Package test suite runtime
zstd, before fix	~22	9.15 s
zstd, after fix	~604	0.02 s
Improvement	~27x	~450x

12.4 Memory behaviour

Seal and restore both stream in bounded chunks, so peak plaintext memory equals one chunk regardless of payload size. A 32 MiB payload and a 32 GiB payload impose the same memory ceiling. No intermediate archive is ever materialised on disk or in RAM.

12.5 Where the time goes

Capsule startup is dominated by Argon2id. This is deliberate: it is the memory-hard passphrase cost that makes offline search expensive, not an inefficiency to be optimised away. Everything else — fingerprint capture, integrity commitments, environment probes, key composition — is measured in microseconds.

13. Security Analysis and Adversarial Review

13.1 Threat model

The attacker possesses the capsule and full knowledge of this architecture — Kerckhoffs's principle applied without exception. They can copy, disassemble, debug, patch, instrument, hook, snapshot memory, run repeatedly, manipulate the clock, virtualise, and inspect every syscall and file the capsule touches.

They do not possess the passphrase, an authorized machine, or — where those protections are enabled — the authorized region or time window.

13.2 Adversarial review results

Attack question	Answer	Basis
Can they extract the payload key?	No, not from the capsule alone	Key is HKDF over factor contributions; capsule holds no key material
Can they bypass fingerprint authorization?	Partially — weakest link	S_machine is extractable from a held capsule; passphrase protection unaffected
Can they patch one branch?	Yes, and it does not help	Demonstrated experimentally in 10.4
Can they force manual location or time?	No manual path exists	Both sides fail closed; no entry field anywhere
Can they set the clock?	Yes	Local clock is attacker-controlled; policy control, not expiry
Can they replace the fingerprint list?	No, not usefully	Sealed and AAD-bound; a forged entry pairs with a useless new secret
Can they transplant a payload?	No	Two independent mechanisms, verified in 10.5
Can they bypass integrity?	Advisory layers yes, AEAD no	Authoritative check requires the key
Can they recover plaintext from temporary files?	None are created	Direct streaming; every negative test asserts an empty output
Can they exploit logs?	No secrets logged	No passphrase, key, coordinate or raw identifier reaches any output
Can they exploit build artifacts?	Not without detection	Provenance verification against a pre-dispatch commitment
Can they observe sensitive memory?	Yes, with live access	Zeroization minimises lifetime; it is not a defence against a debugger
Can they forge authorization state?	The boolean yes, the material no	Forging 32-byte contributions is the actual requirement

13.3 What an attacker recovers immediately

This section is deliberately unflattering Any serious evaluation should begin here rather than with the capability list. These are the facts about what possession of a capsule yields.

Recoverable from a held capsule	Consequence
K_bootstrap and C_runtime (embedded constants)	Can unseal the policy record

Recoverable from a held capsule	Consequence
Policy record: fingerprint digests and S_machine secrets	Obtains the machine factor for capsules already held
Package salt, id, policy flags, Argon2 parameters	Public by design
All code paths and the state machine	Assumed public

It follows that the machine protection is not a standalone confidentiality boundary. For a capsule in hostile hands it constrains convenience and forces per-capsule reverse engineering; it does not stop a determined analyst on its own. What remains is the passphrase protection, which is why it is memory-hard and mandatory.

13.4 Measured side channel

Protection evaluation is ordered and returns on first failure, so denial latency reveals how far execution progressed. This was measured rather than assumed:

Denial stage	Before	After
Binding or machine failure	~2.0 ms	~105 ms
Passphrase failure	~95-105 ms	~117 ms
Ratio	~50x	~1.1x, within noise

The original separation was distinguishable in a single run without statistics, and told an attacker whether they had cleared the machine protection - exactly what the generic failure message exists to hide.

Authorization is now constant-shape. Every protection is evaluated and key derivation always runs before any verdict is returned; a protection that fails contributes a random decoy of the right shape rather than returning early, so the run continues and fails at authenticated decryption like every other denial. The decoys are random rather than fixed, because a constant would be a recognisable marker in memory and a reused one would let an attacker confirm which protection failed by watching for it.

An intermediate version still leaked A first attempt kept an `authorized` flag and short-circuited on it. That still distinguished causes: flag failures stopped at payload authentication while a wrong passphrase ran on to decryption. The flag was removed from the control flow entirely - there is no branch left to patch, and the cryptography decides. Caught by the test that compares state traces, not by inspection.

The cost, accepted deliberately, is that every denial now pays the memory-hard passphrase cost. That work is what makes the oracle disappear. Verified by a test asserting that six different denial causes produce identical state traces, plus the measurements above.

13.5 Anti-tamper: an honest accounting

A tempting claim would be that anti-debugging results are mixed into key derivation, so patching a check yields the wrong key. That claim would be false, and the design would be broken. Key material must be exactly reproducible on every legitimate run; debugger state, timing and loaded libraries are not reproducible. A key derived from them would lock out honest users on a busy machine and require a fallback — precisely the silent downgrade the design forbids.

Category	Examples	Role
Deterministic commitments	Package bytes, per-build diversifier, runtime commitment	May and do participate in cryptography
Environment observations	Debugger presence, timing anomalies, injection markers	Defense-in-depth only; not key material, not a gate

The environment probes are folded into a tamper accumulator used for control-flow diversification, and are deliberately not consumed as a single "if detected then exit" gate, since one patched branch would defeat that. They raise analysis cost. They do not prevent extraction by an attacker who defeats them, and are not claimed to.

13.6 Defects found and fixed during review

Six substantive defects were discovered by adversarial review and correction passes rather than by test failure. They are recorded because the record of what went wrong is part of an honest engineering account.

Defect	Why it mattered	Resolution
runtime_binding was read only from the package header	The capsule never asserted its own identity; binding rested on a single mechanism	Policy-seal subkey now binds the runtime's embedded commitment
Builder capability compiled into every capsule	Package-construction code shipped into hostile-environment binaries	Moved behind a non-default feature; verified absent
Crypto core had an unconditional OS dependency	The auditable boundary carried platform responsibility	Randomness injected via trait; builds with zero platform surface
Time windows embedded the absolute day	A "daily" schedule would have opened on one day only	Made day-invariant; regression test opens a year later
Hardening docs claimed probes fed key derivation	False, and would have produced a broken design	Corrected to separate deterministic commitments from advisory probes
No artifact provenance existed	A hash match does not prove an artifact belongs to a build	Build manifest, signed input, replay resistance

13.7 Outstanding limitations

#	Limitation	Prototype-I status	Future integration
1	S_machine is extractable from a held capsule; the machine protection is not standalone	Needs hardware attestation	Partly — a server key share adds a factor absent from the binary. Hardware attestation remains the real fix and is independent of licensing
2	Protection-order timing side channel	Fixed: ~50x reduced to ~1.1x	No longer a dependency for licensing work
3	Local clock is attacker-controlled	Needs authenticated time	Yes — authenticated server time
4	Location and time are low-entropy policy factors, not entropy sources	Inherent	No — inherent to what they are
5	No anti-hooking or control-flow flattening	Omitted deliberately	No — unrelated
6	Zeroization does not defeat live memory capture, swap or core dumps	Inherent	No — outside what software can prevent
7	Self-destruction cannot guarantee erasure on SSD or copy-on-write filesystems	Inherent	No — but revocation makes a surviving capsule unopenable, which is the stronger control
8	No revocation once a capsule is distributed	Not possible standalone	Yes — withholding the key share. Already-extracted plaintext still cannot be recalled

The one that gets worse Row 2 deserves emphasis. Adding a network round trip in front of the protection sequence widens the measured oracle, because a server denial returns before Argon2id runs. The License Server roadmap

therefore places constant-time protection evaluation in Prototype-I as work item zero, before any licensing work begins. Fixing it afterwards would mean shipping a known and worsened weakness inside a paid product.

On the word "unbreakable" NYEDArch is never described as unbreakable, impossible to reverse engineer, or mathematically immune to attack. The accurate formulation is that it raises the cost, difficulty and expertise required for unauthorized extraction. Every mechanism in Section 13.5 is bypassable by a competent analyst given the binary, time and tooling.

14. Forward Architecture — NYEDArch License Server

Scope boundary This section is architecture and design intent only. No part of the License Server is implemented, scaffolded, stubbed or simulated in the current codebase. Prototype-I is a standalone, unlicensed variant intended for internal reference, architecture development and demonstration. Items marked OPEN await product decisions and are not assumed here.

14.1 Why it exists

Prototype-I has structural limitations that no amount of local engineering can resolve, because they require an authority outside the capsule. The License Server is that authority.

Prototype-I limitation	Future integration	What changes	What remains
Local system clock is attacker-controlled	Authenticated server time	Time windows become resistant to local clock manipulation	Network delay and skew must still be handled honestly
No revocation once a capsule is distributed	Centralized revocation	A compromised capsule can be denied server authorization	Already-extracted plaintext cannot be recalled
No account identity or MFA	Mandatory account identity with mandatory MFA	Operator identity becomes verifiable and recoverable	Endpoint compromise remains possible
No authenticated audit history	Signed, ordered capsule lifecycle events	Authorization attempts become visible to the owner	Events the capsule never sends cannot be received
No recovery when hardware changes	Controlled failsafe with MFA plus email approval	A legitimate user regains access after a hardware change	Recovery is an attack surface and must be rate-limited
No centralized misuse detection	Risk scoring across accounts and capsules	Patterns invisible to a single capsule become detectable	Detection is probabilistic, never guaranteed

14.2 Fixed decisions carried into the design

The following are settled product decisions and are treated as constraints rather than options:

- Internet connectivity is mandatory for the first licensed version. Both the Builder and the capsule fail closed without it. No silent offline fallback and no grace period.
- A capsule requires its own runtime authorization relationship with the server. It is not sufficient that the Builder held a license at creation time.
- The server must contribute meaningful authorization material or cryptographically authenticated policy — never a boolean that a patched branch can replace.
- The server must never hold plaintext payloads or user passphrases. The Prototype-I confidentiality model is preserved, not replaced.
- MFA enrolment is mandatory before an account becomes fully operational, using standards-based authenticators, with passkeys and WebAuthn preferred over TOTP alone.
- There is no free tier. Terms are three months, six months and one year, with renewal required.
- A license key is displayed exactly once and is never retrievable in plaintext afterwards.
- A revoked license stays revoked permanently; remaining term may be transferred to a replacement depending on the reason for revocation.
- Temporary restrictions escalate exponentially on repeat — for example one week, then one month — which raises the cost of brute-force attempts.

14.3 The capsule authorization protocol

The naive design — POST a license check and trust a returned boolean — is explicitly rejected, for exactly the reason the Prototype-I branch-patching experiment demonstrates. The server must participate in the key path.

Capsule startup

- > runtime integrity and security initialization
- > authenticated connection to NYEDArch
- > capsule identity verification (capsule, package, runtime identity)
- > license and entitlement verification
- > server-side policy evaluation
- > authenticated authorization response (server challenge/response, replay-resistant, freshness-bound, policy-version-bound)
- > local authorization factors (machine, passphrase, location, time)
- > payload key release / unwrapping

The architectural requirement is that the server contributes a cryptographic component to the composed key rather than a permission flag, so that patching the response handler yields no payload — the same principle already demonstrated locally in Section 11.4.

14.4 The failsafe path

A legitimate user whose hardware changed, or who is working on a trusted machine that was never enrolled, needs a route that does not weaken the model. The intended design combines: a valid license, an MFA code, and an out-of-band email approval by the account owner, after which the local factors must still be satisfied.

Design tension to resolve Recovery is the highest-value target in any system with mandatory MFA. A failsafe that bypasses the machine protection must not become a general-purpose bypass. Delayed approval windows, cancellation periods, notification to all registered contacts, and exponentially escalating restrictions on repeated attempts are the intended counterweights.

14.5 The enterprise scenario this enables

The fingerprint import, export, labelling and search facilities exist so that the same product serves an individual with two machines and an enterprise with thousands. The HR example is illustrative of the intended composition of protections:

An organisation seals employee records into a capsule whose trusted fingerprint set contains the registered company machines of HR personnel plus the single employee to whom the record belongs. The location protection is set to the office premises and the time protection to working hours. The result is that the record cannot be opened outside the office, outside working hours, or on any machine outside that set — including by the organisation's own executives. This addresses insider and outsider exfiltration in the same construction, which is difficult to achieve with server-mediated DRM alone.

14.6 Honest limits of the licensed architecture

- Self-deletion requested is not self-deletion cryptographically confirmed. If a capsule destroys itself before its final event is acknowledged, the server cannot honestly claim confirmation.
- An audit log stored in a database is not immutable merely because it is in a database. Tamper-evidence must be a specific cryptographic construction with stated guarantees.
- A License Server does not eliminate endpoint compromise. A compromised Builder machine remains compromised.
- Risk detection is probabilistic. No automatic response should follow from a single heuristic.

14.7 Settled: server key share and expiry semantics

Two decisions previously carried as open have been settled by the product owner. Both are recorded here with their consequences, because each converts an availability property into a data-availability property and that must be visible in the architecture rather than discovered during operations.

Decision	Ruling	Direct consequence
Server contribution to the payload key	A true key share	The server participates materially in key composition. A patched response handler yields no payload — the intended property. It also means no capsule can be opened while the server is unreachable
Behaviour on license expiry	Capsules stop opening	Entitlement is continuously enforced, not merely at build time. An expired licence denies access to previously created capsules

14.7.1 What the key-share decision imposes on the server

Holding a key share is a materially stronger obligation than issuing signed policy, and the architecture must be built for that from the start:

- The share database becomes the highest-value asset in the system. A breach yields one authorization factor for every capsule ever issued — not plaintext, since the passphrase and machine factors remain, but it removes a layer for the entire installed base at once.
- Shares must therefore be wrapped by an HSM or managed KMS, never stored in plaintext in an application database, and never exportable in bulk by any single operator role.
- Availability becomes a data-availability guarantee. Redundancy, geographic replication and rehearsed failover are correctness requirements, not operational polish. Scheduled maintenance must drain traffic to standby capacity before any change.
- Replication propagates the most sensitive material by design, so every replica inherits the full protection requirement — replicas cannot be treated as lower-tier infrastructure.
- Share loss is unrecoverable by construction. Backup and restore of the share store must be proven by rehearsal, because there is no cryptographic path back.

Consequence to state to customers plainly Under these two rulings, continuity of the NYEDArch service is a precondition for continued access to sealed data, and an expired licence denies access to capsules that already exist. This is a defensible commercial model, but it must be disclosed prominently in the EULA and purchase flow rather than discovered at renewal. Customers holding long-lived archives should be advised to extract before expiry, and an explicit pre-expiry notification sequence is a product requirement rather than a courtesy.

14.8 Tier model

No free tier. Terms are three, six, or twelve months, renewable; there is no perpetual licence.

	Seal (base)	Seal Pro	Vault (team)	Vault Enterprise
Machine protection	Yes	Yes	Yes	Yes
Passphrase protection	Yes	Yes	Yes	Yes
Time protection	—	Yes	Yes	Yes
Location protection	—	—	Yes	Yes
Cryptographic strength	Full	Full	Full	Full
Org management	—	—	Yes	Yes

	Seal (base)	Seal Pro	Vault (team)	Vault Enterprise
Failsafe approvers	Owner	Owner	Admin + optional	Admin + required
Audit retention	Configurable, short	Configurable	Configurable, long	Configurable, longest

14.8.1 Why gating time and location does not breach the no-upsell principle

The governing principle is that core security must not be sold as an upgrade. That principle is honoured, and the reasoning must be stated in exactly these terms rather than blurred into "more security at higher tiers":

- Every tier receives identical cryptographic strength — same AEAD, same Argon2id parameters, same key composition, same integrity and binding.
- Both mandatory protections, which are the actual confidentiality boundary, are present at every tier.
- Location and time are low-entropy policy factors, documented as such throughout Prototype-I. They constrain where and when; they add no meaningful entropy and do not strengthen the cryptography.

A base-tier customer is not receiving weaker encryption. They are receiving fewer policy controls. If that framing were ever contradicted — for example by weakening KDF parameters at lower tiers to reduce server cost — the principle would be broken, and it must not be.

14.9 Remaining settled decisions

Question	Ruling	Consequence
Telemetry retention	User-configurable, bounded per tier	Shorter is the default; fixed floors exist only where an active investigation requires them, and are disclosed rather than silently applied
Failsafe approval, individual	Account owner only	Licence + MFA + out-of-band email approval; local protections still apply
Failsafe approval, enterprise	Team admin + optional additional reviewers	Configurable per organisation, so regulated tenants can require two or three approvers while small teams require one
Mandatory-online scope	Every capsule execution	Follows necessarily from the key-share ruling: the share is required at each key composition, so there is no activation-only variant

The failsafe substitutes for the machine protection only. The passphrase remains required, and location and time remain in force where enabled — it is not a master key. Its honest weakness is that a sufficiently complete account compromise, covering password, MFA and email, defeats it. The counterweights are notification, delay and exponential escalation, not a claim of impossibility.

14.10 Documentation set

The full future architecture is drafted as sixteen documents under docs/license-server/, covering the overview, service architecture, identity and MFA, licensing and entitlements, Builder integration, capsule authorization, the capsule-to-server protocol, audit and forensics, security monitoring, key management, distribution and updates, payments, threat model, incident response, privacy and data model, and the integration roadmap. No part of it is implemented.

Sequencing dependency The roadmap places one item before licensing work begins: fixing the measured protection-order timing oracle in Prototype-I. Adding a network round trip in front of an existing oracle widens it, because a server denial returns before Argon2id runs. Fixing it afterwards would mean shipping a known weakness inside a paid product.

15. Originality and Intellectual Property Record

No legal claim is made This section records engineering provenance to support intellectual-property work. It does not assert patentability, and novelty of appearance is not treated as evidence of novelty in law.

15.1 Established primitives, used as such

NYEDArch invents no cryptographic primitive. XChaCha20-Poly1305, Argon2id, HKDF-SHA512, HMAC-SHA512, SHA-256 and zstd are established, widely reviewed constructions used through maintained implementations. Inventing a primitive to appear novel would be a defect, not a contribution.

15.2 Where the specific work lies

Area	The specific construction	Distinguishing property
Multi-factor key composition	Domain-separated per-factor contributions concatenated as HKDF input keying material, with presence strictly matched to policy	Bypassing a branch cannot synthesise a missing contribution; demonstrated experimentally
Fingerprint bootstrap resolution	A fingerprint selects a minted high-entropy secret rather than deriving a key from machine data	Avoids the circular dependency of needing the fingerprint to decrypt the fingerprint list
Package-to-runtime binding	Policy-seal subkey derived from bootstrap key concatenated with the runtime's own embedded commitment	Transplantation fails cryptographically, not by a comparison that can be patched
Adaptive fingerprinting with disclosure	Strength-tagged signal discovery with recorded availability inventory	Degrades on capability-poor machines without silently substituting weak evidence for strong
Deterministic geographic quantization as a policy factor	Latitude-scaled grid cell with accuracy rejection below configured tolerance	Refuses to accept a reading that cannot prove the claimed region
Day-invariant recurring time windows	Window identifier over time-of-day, tolerance and offset, excluding the absolute day	A recurring policy genuinely recurs rather than silently becoming single-day
Artifact provenance for untrusted build environments	Pre-dispatch build manifest with a signed input covering build, target and three commitments	Establishes that an artifact is the output of a specific build, not merely that bytes match
Capability isolation by build feature	Package-construction code compiled out of the distributed artifact	The hostile-environment binary cannot be repurposed as a packer

15.3 Reference material and independence

An existing project was consulted strictly as a technical reference for GitHub-based build orchestration concepts — authentication, repository initialization, Actions triggering, live logs and artifact retrieval. Its user interface, visual identity, terminology, layout, navigation and interaction model were not used. No code or dependency from it exists in NYEDArch. The orchestration layer was redesigned around NYEDArch's own security model, including artifact provenance, which the reference concept does not address.

The user interface, terminology and design language were independently conceived for NYEDArch.

15.4 Decision record

ADR	Decision	Category
0001	XChaCha20-Poly1305 selected for payload AEAD	Security
0002	Toolchain pins (superseded by 0005)	Implementation
0003	Geographic quantization scheme	Security
0004	Architecture correction pass — eight structural corrections	Architecture / Security
0005	Toolchain upgrade to Rust 1.91.1 and zstd adoption	Implementation / Performance

16. Status, Roadmap and Conclusion

16.1 Phase completion

Phase	Status	Evidence
1 Architecture and threat model	Complete	Architecture and threat model documents
2 Cryptographic and package specification	Complete	Crypto format and key hierarchy documents
3 Workspace and module boundaries	Complete	8 crates with enforced feature separation
4 Package, encryption, fingerprint	Complete	Round-trip and fingerprint tests
5 Runtime generation	Complete	Capsule compiled and executed
6 GUI	Code complete	Compiles with zero warnings; needs a display to render
7 GitHub orchestration	Complete	Transport verified against the live API
8 Cross-platform	Verified on all three	Test suite, fingerprinting, capsule build and run, key storage, build scripts, installer and the macOS bundle all exercised on real runners
9 Hardening	Complete	Anti-RE analysis, tamper accumulator
10 Adversarial review	Complete	15 questions; 6 defects found and fixed
11 Performance	Complete	Measured; one defect found by measurement
12 Documentation	Complete	14 documents plus this dossier

16.2 What has not been executed

Stated explicitly so that no reader over-reads the verification status:

- The macOS location permission prompt is still unverified: CoreLocation compiles and the bundle carries the usage description, but a headless runner has no location services and nobody to grant permission. That needs a Mac with a user in front of it.
- macOS CoreLocation now compiles on a real macOS runner and produces a capsule. What remains unverified is runtime behaviour: that the permission prompt appears and returns a fix on a Mac with the bundle installed.
- No release-mode desktop binary is shipped: the link-time-optimised build exceeds the time budget of the build environment used here. Both build scripts include it automatically where it completes.
- The GitHub transport is verified against the live API; a complete authenticated build requires an account and token.
- TPM and Secure Enclave attestation are designed for but not implemented.

16.3 Building and distributing

The repository ships a single build script. It verifies the toolchain, runs the full test suite, refuses to produce a release from a failing build, compiles the workspace and desktop client, confirms the cryptographic core still builds with zero platform surface, measures performance on the build machine, and assembles a release directory.

```
./build.sh          # test, build, benchmark, assemble ./release
./build.sh --skip-gui # omit the desktop client
./build.sh --clean   # discard previous artifacts first
```

```

release/
  bin/nyedarch          command-line client
  bin/nyedarch-gui      desktop client with drag-and-drop launcher
  runtime-src/          runtime crates vendored into generated capsules
  docs/                 architecture, security, threat model, EULA
  docs/license-server/  future architecture (design only)
  START_HERE.md         end-user guide
  BUILD_INFO.txt        toolchain, revision, test result
  SHA256SUMS            integrity checksums

```

A distribution defect found by testing the release, not the source Generated capsule projects originally referenced the NYEDArch crates by absolute path on the machine that produced them. Everything worked in development and would have failed for every real user — and equally on the GitHub runner, which has no NYEDArch source tree. Generated projects now vendor the six runtime crates and declare their own workspace root, so they are self-contained. Verified by hiding the source tree entirely and building and running a capsule from the release directory alone.

16.4 Engineering priorities

Priority	Work	Addresses
1	Hardware-attested machine identity (TPM sealing, Secure Enclave)	The S_machine extractability weakness — the most significant open item
2	Hardware-attested identity follow-through (TPM sealing, Secure Enclave)	The S_machine extractability weakness
3	Platform execution on macOS and Windows	Completes cross-platform verification
4	Authenticated time source	Local clock trust
5	License Server implementation (architecture drafted — see Section 14 and docs/license-server/)	Revocation, identity, authenticated time, audit, recovery

16.5 Conclusion

NYEDArch demonstrates that authorization policy can travel inside data as a cryptographic property rather than as a procedural check enforced by external infrastructure. The central claim — that bypassing a conditional branch does not yield the payload — was substantiated by removing the branch, recompiling, and observing that extraction still failed.

The prototype is complete and functional: 53 tests pass, the full pipeline executes end to end, throughput is measured rather than asserted, and the capsule produced is a real executable that restores its payload byte-identically on an authorized machine and refuses everywhere else.

Equally, this document records what does not work. The machine protection is not a standalone confidentiality boundary. A timing oracle exists and is measured. The clock is trusted. Three platforms are designed for and one is exercised. These are stated because a security product whose documentation cannot be trusted is worth less than one with fewer features and an honest account of them.

The forward architecture is drafted to the same standard. Sixteen documents specify the License Server that adds revocation, authenticated time, identity, audit and recovery — and they are equally explicit that mandatory connectivity trades away the offline property, that expiry denies access to capsules that already exist, and that one existing weakness gets worse before it gets better. None of it is implemented; Prototype-I remains standalone.

Final statement NYEDArch raises the cost, difficulty and expertise required for unauthorized extraction. It does not make extraction impossible, and no claim to the contrary is made anywhere in this document or in the implementation.